

CONTROL DE VERSIONES PROFESIONAL

Git Esencial: Aprende los Comandos Básicos

Guía Práctica e Intuitiva para Dominar el Flujo de Trabajo y la
Arquitectura de Git

ESCRITO POR

Johan Mancebo

@JohansitoDev

DESARROLLO DE SOFTWARE • GESTIÓN DE CÓDIGO • EDICIÓN 2026

Índice General

Introducción General	1
Capítulo 1: Configuración Inicial e Inicialización	2
1.1 Configurar tu Identidad Global en el Sistema	2
1.2 Inicializar un Repositorio (<code>`git init`</code>)	2
1.3 Clonar Repositorios Existentes (<code>`git clone`</code>)	3
Capítulo 2: El Ciclo de Vida de los Archivos (Los 3 Estados)	4
2.1 Working Directory, Staging Area y Repository	4
2.2 Verificar el Estado Actual (<code>`git status`</code>)	4
2.3 Rastrear Archivos de Forma Selectiva (<code>`git add`</code>)	5
Capítulo 3: Confirmaciones e Historial	6
3.1 Crear Instantáneas de Código (<code>`git commit`</code>)	6
3.2 Escribir Mensajes de Confirmación Atómicos	6
3.3 Inspeccionar la Línea de Tiempo (<code>`git log`</code>)	7
Capítulo 4: El Poder de las Ramas (Branching)	8
4.1 Crear y Moverse Entre Ramas (<code>`git branch`</code> y <code>`git checkout`</code>)	8
4.2 Fusionar Características (<code>`git merge`</code>)	8
4.3 Entender y Resolver Conflictos Básicos de Fusión	9
Capítulo 5: Sincronización con Repositorios Remotos	10
5.1 Vincular un Servidor Remoto (<code>`git remote add origin`</code>)	10
5.2 Subir e Implementar Cambios (<code>`git push`</code>)	10
5.3 Traer y Fusionar Actualizaciones (<code>`git pull`</code> vs <code>`git fetch`</code>)	11

La Revolución del Control de Versiones Distribuido

En el desarrollo de software moderno, el código es un ente vivo. Cambia, muta y se expande a cada minuto. Trabajar sin un sistema de control de versiones es el equivalente a construir un rascacielos sin planos ni la posibilidad de deshacer un error estructural. Aquí es donde **Git** se consolida como el estándar absoluto de la industria.

"Git no es una simple herramienta de respaldo; es una máquina del tiempo exacta que registra la evolución intelectual de un proyecto, permitiendo la colaboración asíncrona y masiva sin riesgo de destrucción mutua del código."

Creado en 2005 por Linus Torvalds, Git cambió el paradigma de los sistemas centralizados. Al ser distribuido, cada desarrollador posee una copia local completa de toda la historia del código. Esta guía práctica está diseñada bajo mi perspectiva metodológica: directa, técnica y enfocada en los comandos esenciales que utilizarás el 99% de tus días como desarrollador. Dominar estos comandos básicos te dará la confianza necesaria para experimentar y colaborar sin miedo.

Configuración Inicial e Inicialización

1.1 Configurar tu Identidad Global

Antes de realizar cualquier confirmación (commit), Git necesita saber exactamente quién es el autor del cambio. Esto es crucial en entornos profesionales y de auditoría de código. Configuramos el nombre y correo con los siguientes comandos desde la terminal:

```
# Configura el nombre del autor universal en la máquina
git config --global user.name "Johan Mancebo"

# Configura el correo asociado
git config --global user.email "johan@example.com"
```

1.2 Inicializar un Repositorio local (`git init`)

Para transformar una carpeta común del sistema operativo en un repositorio inteligente rastreado por Git, debemos posicionarnos en ella vía terminal y ejecutar la inicialización. Esto crea un directorio oculto llamado `.git`.

```
cd ruta/de/tu/proyecto
git init
```

1.3 Clonar Repositorios Existentes (`git clone`)

Si el código ya existe en servidores remotos (como GitHub o GitLab), descargamos una copia exacta junto con todo su historial de desarrollo utilizando la clonación:

```
git clone https://github.com/usuario/repositorio.git
```

El Ciclo de Vida de los Archivos

2.1 Los Tres Estados (Áreas) de Git

Para entender Git, es indispensable asimilar que los archivos pasan por tres zonas conceptuales claramente diferenciadas antes de guardarse en el historial:

- **Working Directory (Directorio de Trabajo):** Tus archivos locales en tiempo real, donde editas y escribes código. Las modificaciones aquí están en estado *Modified* (Modificadas) pero no aseguradas.
- **Staging Area (Área de Preparación / Índice):** Una zona intermedia. Aquí colocas los cambios específicos que decides que formarán parte de tu próxima instantánea. Los archivos están en estado *Staged*.
- **Repository (Directorio Git):** Donde los cambios se guardan permanentemente en la base de datos local en forma de *Commits*. Estado *Committed*.

2.2 Verificar el Estado Actual (`git status`)

Es el comando más utilizado del día a día. Te dice de forma explícita qué archivos han cambiado, cuáles están listos para ser preparados y cuáles no están siendo rastreados por Git:

```
git status
```

2.3 Rastrear e Indexar Archivos (`git add`)

Para mover un archivo modificado desde tu directorio de trabajo hacia el Staging Area, usamos el comando de adición:

```
# Añade un archivo específico
git add index.html

# Añade ABSOLUTAMENTE todos los cambios de la carpeta actual
git add .
```

Confirmaciones e Historial

3.1 Crear Instantáneas de Código (`git commit`)

El commit consolida los cambios que se encuentran actualmente en el Staging Area y los guarda de forma permanente en la línea de tiempo. Cada commit genera un hash SHA-1 único que lo identifica.

```
git commit -m "feat: implementar barra de navegación responsive"
```

3.2 Mensajes de Confirmación Atómicos

Un buen desarrollador escribe mensajes descriptivos. La recomendación metodológica es usar el modo imperativo y prefijos claros (como `feat:` para nuevas características, `fix:` para corrección de bugs, o `docs:` para documentación). Evita mensajes ambiguos como "cambios realizados" o "arreglos varios".

3.3 Inspeccionar la Línea de Tiempo (`git log`)

Para auditar el historial de confirmaciones, ver quién hizo cada cambio, la fecha y los mensajes descriptivos, consultamos el registro histórico:

```
# Muestra el historial completo
git log

# Muestra una versión compacta y legible en una sola línea por commit
git log --oneline
```

El Poder de las Ramas (Branching)

4.1 Crear y Moverse Entre Ramas

Las ramas permiten ramificar la línea de tiempo principal (generalmente llamada `main` o `master`) para desarrollar nuevas funcionalidades de forma aislada, sin romper el código que ya funciona en producción.

```
# Crear una nueva rama llamada 'desarrollo-login'
git branch desarrollo-login

# Cambiar de rama activa para empezar a trabajar en ella
git checkout desarrollo-login
```

4.2 Fusionar Características (`git merge`)

Una vez que tu código en la rama secundaria funciona perfectamente y pasa las pruebas, debes integrarlo a la rama principal. Para ello, primero te paras en la rama que va a recibir los cambios y luego ejecutas el merge:

```
git checkout main
git merge desarrollo-login
```

4.3 Resolver Conflictos de Fusión

Si dos ramas modificaron exactamente la misma línea de un mismo archivo, Git no sabrá de forma automática cuál versión conservar y generará un **Conflicto**. Git pausará la fusión y marcará el archivo con delimitadores visuales. El desarrollador debe abrir el archivo manualmente, elegir la versión correcta, borrar los delimitadores, añadir el archivo al Staging y hacer un nuevo commit.

Sincronización con Repositorios Remotos

5.1 Vincular un Servidor Remoto

Si creaste un repositorio en tu máquina y quieres subirlo a plataformas como GitHub, primero debes agregar la URL del servidor remoto asignándole un alias, que por convención estándar es `origin`:

```
git remote add origin https://github.com/usuario/proyecto.git
```

5.2 Subir Cambios al Servidor (`git push`)

Para enviar tus commits guardados localmente hacia la nube/servidor remoto para que tu equipo pueda verlos, empujamos la rama correspondiente:

```
git push -u origin main
```

5.3 Traer y Fusionar Actualizaciones (`git pull`)

Si tus compañeros de equipo subieron nuevos cambios al servidor, necesitas actualizar tu copia local. `git pull` descarga los cambios remotos y los fusiona de inmediato en tu rama activa actual:

```
git pull origin main
```

Nota técnica: `git pull` es en realidad la combinación secuencial de dos comandos intermedios: `git fetch` (que solo descarga los datos para inspección) y `git merge` (que realiza la fusión).